nleFlow: plan and recommendation

nleFlow: plan and recommendation

Planning stage for AH, 2026-10-16, from `handover/NLEFLOW_PLAN_PROMPT.md`. Branch `nleflow-plan`. Everything stated about the code was read from the files cited; everything stated about what runs was run on this cloud instance (Ubuntu 24.04, 4 cores, 15 GB, R 4.3.3) and is reported in §9.5 with what was not run.

0. Summary and recommendation

Question. Is it worth building a separate R package, `nleFlow`, that (1) learns a new neural likelihood flow by our recipe, (2) exports flows and LANs to EMC2 cards and to ONNX, and (3) converts ONNX LANs to EMC2 cards?

Recommendation: build a smaller thing, in two halves, and no R package called nleFlow.

1. **A Python package `nleflow`** for the parts that are Python by nature: a model specification, one training command, the checkpoint exporter with golden vectors and the float64 gate, the ONNX exporter (including the DDMnle graph that does not exist yet), and the ONNX-to-card converter (today an EMC2 `inst/scripts` file that needs Python anyway). This is where "learn a new flow" lives, and it is a generalisation of `handover/nle-methods/flow/`, not a rewrite.
2. **The R side stays in EMC2 `dev-nle`**, which already holds the card loader for four kinds of network, `register_nn_model()`, the validation kit (`nn_cell`, `nn_in_box`, `nn_total_mass`, `nn_score_bias`, `nn_posterior_shift`, `nn_sbc_cell`), the converter, a vignette and 500+ passing tests (§1.3). What EMC2 lacks is small and specific: a versioned, written card spec; the DDMnle (scaled) kind wired in R, not only in C++; a golden-vector check; a fix for R < 4.5 (§1.3). The nle repo's R reference evaluators (`port/R/flow_race.R`, `flow_ddm.R`, `ddm_scaled.R`) become the fixtures EMC2's tests pin against, as they already are for two of the three.

Why not an R package that trains. Training is JAX. Wrapping it in R through reticulate adds the heaviest dependency in the project (a pinned JAX environment, GPU optional) to an R package that could not go to CRAN and would duplicate EMC2's card loader, which is the one piece that must not drift. An R-only package that only exports, converts and validates would duplicate what `dev-nle` has already, with tests, and would re-create the "two copies of the same evaluator" trap that cost this project weeks (`handover/EMC2_IMPLEMENTATION_BRIEF.md:15-19`).

Why a Python package rather than the scripts as they are. Nothing in `flow/` is general: every context list, transform, box, zero-box constant and output path is a literal in a script (§4.2, §8). A user with a new model edits six files. The two worked examples, DDMnle and the RDM, were each built by adding a flag and a context list to those files; a third model would be a third flag. A model specification plus one command is the smallest change that makes the recipe usable by someone else, and it is also what makes the two examples reproducible as tests.

Audience, honestly. The users are us first (Julian, Malte, AH: the paper's reproducibility and the next models), EMC2 users with an intractable model second (a handful of labs), and HSSM users wanting EMC2 third (they need only the converter, which exists). That audience justifies a lean package with two worked examples and a verification command, not a product.

Smallest useful first milestone (about a week, §11): the card spec v1 written down and versioned; the DDMnle kind wired in R inside EMC2 (today reachable only through `port/R/ddm_scaled.R` plus a scaffold card that is not committed); `nn_golden_check()`; golden vectors regenerated for the committed DDMnle card (its golden file does not exist, §2.4); and one verification command that passes on a fresh machine. Nothing in that milestone depends on deciding the Python package.

Decisions AH needs to make are listed in §11.3.

---

1. What exists today, verified here

1.1 EMC2 `dev-nle` (the tarball, 142bd497)

`handover/emc2/emc2_142bd497.tar.gz` is `git archive` of `dev-nle` at 142bd497, `Version: 3.4.1.9001` (`DESCRIPTION:3` of the extracted tree; `handover/emc2/README.md:4` says "3.4.1", which was the dev-nn era build). It is the final sampler (`dev-sampler` aa66e11d merged into `dev-nle`, `handover/MALTE_STAGES.md:1106-1108`).

It is less "not on GitHub" than the brief says. `ampl-psych/EMC2` has a `dev-nle` branch at 154f21ad (2026-10-02, the `qbinom` fix) on top of 14c12c79 (2026-09-28, natural-scale race flows, hybrid race, CRDM) and d49d190b (zero-box). So the whole NN layer (cards, `register_nn_model`, the kit, the converter, the tests) is public. What is not pushed is the merge with the final sampler (142bd497) and the scaled-DDM patch (`handover/emc2/emc2_dev-nle-scaled.patch`). Meanwhile `dev` on GitHub has the new sampler (8b9bdbf, 2026-10-06) and later commits, so `dev-nle` is now behind `dev` on the sampler and ahead of it on NN: a rebase is due before any of this work lands.

What the tree contains (file:line in the extracted EMC2):

piece	where	notes
card kinds	`R/nn_register.R:55` `nn_kinds <- c("flow_joint","flow_race","regression_joint","mlp_joint")`	inferred fr (:208-213) flow_mlp+ → flow_join → flow_rac == "mlp_j other mlp -

piece	where	notes
`register_nn_model(path, pars, transforms, kind, cdf, exceptions, pre, twin, p_types, hybrid)`	`R/nn_register.R:503-505`, roxygen `:240-502`	`.rds`, `.jso` jsonlite) or name; cont asserted ag `(:521-536)` must agree every twin be an inpu or fixed (`:6`
transforms	`:57` `identity, log, probit, zerobox`; zerobox = asinh(x/c) with `zerobox_c` per input (`:66-93`)	
compiled evaluators	`src/model_NN.h` (dispatch, `NnSpec` `:39-60`), `src/flow_ddm.cpp`, `src/flow_race.cpp`, `src/mlp_lik.cpp`, `src/nle_flow.h`	refuses cer truncation (`model_NN` kinds are t `(:165-169)`
validation kit	`R/nn_validate.R`: `nn_cell` `:94`, `nn_in_box` `:326`, `nn_total_mass` `:621`, `nn_score_bias` `:734`, `nn_posterior_shift` `:876`, `nn_sbc_cell` `:1047`, `nn_sbc_summary` `:1101`	`run_sbc` re that leaves (`R/SBC.R:1`
converters	`inst/scripts/onnx_to_card.py` (MLP LANs, tanh only, rank-1 input, one preset `ddm_uniform_st`), `inst/scripts/regress_to_card.R`	
gates and scripts	`inst/scripts/nle_acceptance.R`, `nle_validation_kit.R`	the accepta compares s with the N 10
docs	`man/register_nn_model.Rd` (the only description of the card fields, roxygen `:263-379`), `vignettes/neural-likelihoods.Rmd`	no standalo no version
shipped artefacts	`inst/extdata/flownn/`: `ddm_cap256w_c4.rds`, `ddm_st0zero.rds`, `rdm_small.rds`, `MANIFEST` (sha256)	
tests	`tests/testthat/test-nn-{port,register,mlp,zerobox,malte,native,validate,reg-s402}.R`; golden files under `tests/testthat/golden/` include `ddm_uniform_st.onnx`, `lan_ddm_uniform_st_ort.csv`, `rdm_rdm_small_e195_golden.csv`, `ddm_cap256w_golden.csv`	

The scaled patch (`handover/emc2/emc2_dev-nle-scaled.patch`, 177 lines, `src/model_NN.h`, `src/flow_ddm.cpp`, `src/nle_native.h`) adds `NnSpec::scaled`, the eight DDM columns, `nn_trial_ll_scaled()` and `nle_ddm_native_full()` (pdf and CDF). **It is C++ only**: nothing in `R/` sets `scaled` or `sc_pars`. The R-side wiring is `ddm_sc_native_reg()` in `handover/nle-methods/port/R/ddm_scaled.R:115-135`, which copies a DDM registration built from a zero-box "scaffold" card (`sbc/scaled_cell.R:27,38`: `ddm_zb_s903_e271_weights.json`) and rewrites its native spec. That scaffold card is not in the repository (`find` returns only `.out/.rds` under `sbc/zerobox_postshift_v1/`). See §2.4.

1.2 The Python training and export code

`handover/nle-methods/flow/` (JAX 0.5.3, flax 0.10.6, distrax 0.1.5, optax 0.2.8, orbax < 0.11.2, numpy 2.2.6, Python 3.11–3.12: `pyproject.toml:10-28`, `uv.lock`). `onnx` and `onnxruntime` are imported by `scripts/export_onnx_ddm.py` and `scripts/onnx_check.py` but are **not in the lock or pyproject**. The handover copy and `/home/user/nle/flow` have diverged in both

directions: the handover copy has `--zerobox`, `--scaled` and the float64 gate; the root copy has `--decision-time` for the DDM, a README, `requirements.txt`, notebooks and tests (`diff -rq`; details in §8).

There is no generic entry point. The closest are `scripts/train_race.py` with its `CONTEXT` dict (`:71-74`: RDM `v,B,t0,s,A`; LNR `m,s,t0`) and `scripts/export_flow.py` with `MODEL_REGISTRY` (`:74-94`); the DDM has its own `train_ddm.py`, `retrain_classifier.py`, `export_flow_ddm.py`, each with a `CONTEXT*` literal per build (`export_flow_ddm.py:45-61`).

1.3 What I ran here (fresh Ubuntu 24.04 container, R 4.3.3)

step	result
R 4.3.3 from apt	45 s
16 EMC2 dependencies as `r-cran-*` from apt	2 min 16 s; `WienR` is not packaged, built from the GitHub CRAN mirror in 18 s (CRAN itself is blocked by this environment's network policy)
EMC2 `dev-nle` 142bd497 from the tarball, dedicated library	builds; `packageVersion` 3.4.1.9001; patched copy with `--preclean -j4`: 78 s
Python stack by `uv` (jax 0.5.3, flax 0.10.6, onnx 1.23.2, onnxruntime 1.31.0, numpy 2.2.6)	11 s
`port/tests/test_ddm_scaled.R` (DDMsc oracle: rescaling, t0 shift, st0 convolution vs `dDDM`)	**PASS**, 7 s (max
`flow/scripts/onnx_check.py` on `ddm_cap256w_c4_shared.onnx` and `ddm_cap256w_c4.onnx` against 400 R-port cases	max
`inst/scripts/onnx_to_card.py` on the LAN `ddm_uniform_st.onnx` from EMC2's test tree, `--preset ddm_uniform_st --verify 2000`	writes the card; card vs onnxruntime max
EMC2's own nn tests (`test-nn-port/register/mlp/zerobox/malte/native`)	**503 expectations pass, 10 errors, 3 skips**; every error is one trap: `tools::sha256sum` does not exist before R 4.5, `nle_file_hash` (`R/nn_register.R:42-46`) falls back to md5, and `inst/extdata/flownn/MANIFEST` holds sha256 only, so every *shipped* artefact fails "does not match the sha256 pinned". JSON cards register fine. On R ≥ 4.5 (the laptop runs 4.6.0) this never shows
the committed DDMnle card `port/export/ddm_sc_s1106_e245_weights.json` through `ddm_sc_card_evaluator()`	loads; 0.415 ms per 400-row evaluator call; total mass by quadrature up to log u = 2 is 0.9757 and the sum of the two defective CDFs at the spline's `range_max` is exactly $\Phi(2) = 0.97725$ for every θ tried (§2.4)
the compiled DDMnle path: patched build + `sbc/scaled_cell.R ... BENCH=1 NATIVE=1` with a throw-away scaffold card (§2.4)	compiled vs R path max
the LAN card registered with its twin (`sbc/lan_case5/lan_model.R`)	registers (type DDM, `c_name` NN). My hand comparison with the ORT golden rows returned the floor and I did not chase it; EMC2's own `test-nn-mlp.R` covers that comparison and passes

Not run here: anything on tux26, Gadi or AWS; training; the SBC cells; `nle_acceptance.R` (it loads the shipped artefacts by name and so hits the R 4.3.3 hash trap); `test-nn-validate.R` and `test-nn-reg-s402.R`.

---

2. Worked example A: DDMnle (the scaled DDM)

2.1 How it was built with the current code

Sources: `handover/SCALED_DDM_PLAN.md`, `handover/nle-methods/simulation/scaled.R`, `simulation/sim_ddm_scaled.R`, `flow/scripts/train_ddm.py --scaled`, `retrain_classifier.py --scaled`, `export_flow_ddm.py --scaled`, `port/R/ddm_scaled.R`, `sbc/scaled_cell.R`, `flow/scripts/scaled_pipeline_tux.sh`.

step	what	where
parameterisation	the network sees (κ, η, Z, SZ) with κ = v a/s², η = sv a/s²; it models the unit-scale decision time u = (rt − t0)(s/a)² of a diffusion with a = s = 1; a and s are an exact rescaling, t0 an exact shift, st0 an exact difference of two CDFs: L = [G(u₁) − G(u₀)]/st0; exact zeros η = 0, SZ = 0; st0 = 0 is the plain density	`SCALED_DDM_PLAN.md:11-35, 62-75`; `ddm_scaled.R:1-16, 75-96`
encodings	κ identity; η and SZ `asinh(x/c)` with c = (0.01, 0.05), exact 0 at 0; Z probit	`scaled.R:13`, `ddm_scaled.R:25-40`; the card records `context_transforms = {kappa: identity, eta: zerobox, Z: probit, SZ: zerobox}` and `zerobox_c`
box and regimes	κ ∈ [−10, 10] (half the draws in [−3, 3]); η ∈ {0} ∪ [0.01, 8] (75 % log-uniform, 25 % asinh-uniform); Z ∈ [0.1, 0.9]; SZ ∈ {0} ∪ [0.05, 0.99]; regimes full 55 %, η = 0 15 %, SZ = 0 15 %, both 15 %	`scaled.R:11-25`
simulation	EMC2's `make_data` at a = s = 1, st0 = 0, fixed T0SIM = 0.05; the stored `ll` from `dDDM` at precision 5e-10; each chunk rewritten to columns `param_id, R, rt(=u), ll, kappa, eta, Z, SZ`; 600k draws × 40 rows (seed 5101), validation 25k × 40 (5102); Sobol with a seeded Cranley–Patterson shift	`scaled.R:27-57, 59-65`; `sim_ddm_scaled.R:1-9, 39-41`; 16 min on tux26 (`SCALED_DDM_PLAN.md:132`)
classifier target	exact P(R = 2	κ, η, Z, SZ) from `WienR::pWDM` at a = 1, sw = 2 SZ min(Z, 1 − Z), precision 1e-8
flow	conditional rational-quadratic spline over log u, [256, 128] conditioner, 64 bins, spline range [−12, 2] set from the pilot's data, batch 1024, 300 epochs, 30 checkpoints kept, 6 seeds × 4 threads = 24 cores, 26 h (07:29 14 Oct → 09:21 15 Oct)	`SCALED_DDM_PLAN.md:39-43, 179`; `scaled_pipeline_tux.sh:84-88`
classifier	`clf_sc_big` on the exact targets, mean	err
export and gate	float64 NumPy port of the network produces the golden vectors;	`export_flow_ddm.py:243-260, 425-435`;

step	what	where
	gated against JAX **float64** at 1e-8 (`--ref f64`, the default under `--scaled`), because the float32 JAX reference itself differs from the port by up to 2.8e-2 in the far right tail; 180 cards exported in 5 min, worst 1.8e-10	`SCALED_DDM_PLAN.md:159-181`; the diagnostic `flow/scripts/port_x64_check.py`
screening	score bias in 7 regimes, two stages (100 draws on all 180 cards, then 300 on the leaders); score = Σ over regimes of	g_v
SBC	18 cells (3 cards × 6 models) on AWS, N = 400, 500 replicates, network only, against the existing analytic controls; `sc_s1106_e245` outside the 95 % envelope once in 35 tests → adopted as DDMnle	`SCALED_DDM_PLAN.md:194-226`; `sbc/aws/scaled_sbc/run1/`
evaluation in EMC2, R path	`DDMsc(card)`: `DDM()` with `dfun` replaced, `c_name` dropped; transforms, bounds, zero exceptions and `rfun` are DDM's own	`ddm_scaled.R:98-109`
evaluation in EMC2, compiled path	the patch plus `ddm_sc_native_reg()`; agrees with the R reference to ≤ 1.1e-11 (laptop), 7e-13 here	`ddm_scaled.R:115-135`; `SCALED_DDM_PLAN.md:245-264`

2.2 What the package would need to reproduce it

A model specification (the shape is in §4.3) that says: inputs `kappa, eta, Z, SZ` with those transforms and constants; the box and the regime mixture above; a simulator that is a thin wrapper on EMC2's `make_data` plus the column rewrite; a choice classifier with an exact target function; `pre = "t0"` plus a scaling rule and a CDF-difference rule. The last two are the only genuinely new things in the whole example: the flow and classifier machinery is the same as every other DDM build. On the EMC2 side the card needs one more field than `flow_joint` has today (a `scaling` rule, now a free-text string at `export_flow_ddm.py:464-466`) and the loader needs to read it, which is what the patch does in C++ and nothing does in R.

Commands today: `Rscript simulation/sim_ddm_scaled.R train|val|choice`, `train_ddm.py --scaled ...`, `retrain_classifier.py --scaled --config big --out-json`, `export_flow_ddm.py --scaled --step N`, then `sbc/scaled_screen/run_stage{1,2}.sh`, `sbc/scaled_select.R`, `sbc/scaled_cell.R`.

2.3 Speed, as measured

`SCALED_DDM_PLAN.md:255-264` (laptop, N = 400): compiled DDMnle 0.075–0.125 ms per likelihood against WienR 0.33–1.22, 2.9x–9.8x; this machine 3.8x (full) and 1.6x (st0 only). The report (`REPORT_RUSS_JULIAN.md:22-35`) gives the per-model table. Against a continuous trial-level regressor the flow conditions once per unique row and loses its advantage (`EMC2_INTEGRATION.md:251-288`); nothing in this plan changes that, and the installation docs should say so.

2.4 What is still missing, and what I found

1. **ONNX export of DDMnle does not exist.** `export_onnx_ddm.py` builds the 8-parameter graph; DDMnle needs the network's CDF in the graph (spline CDF and a normal CDF through `Erf`), the preprocessing κ, η, c from the natural parameters, the t0 shift and the st0 branch, and an acceptance test against the compiled path to ~1e-11 (`REPORT_RUSS_JULIAN.md:152-167`). Probit and log stay with the caller.
2. **The committed DDMnle card has no golden vectors.** `port/export/` holds `ddm_sc_s1106_e245_weights.json` (and the two runners-up) but the only scaled golden file is `ddm_sc_pilot_e27_golden.csv`, whose weights are *not* committed; `port/tests/test_ddm_scaled_card.R` therefore cannot run on the committed material (it stops at `normalizePath`). Regenerating `ddm_sc_s1106_e245_golden.csv` needs the checkpoint on tux26 and one `export_flow_ddm.py --scaled --step 245` run. **This is the first thing the verification command needs.**
3. **The compiled path is not reachable from `register_nn_model()`.** It needs the patch (uncommitted on the EMC2 side), `ddm_sc_native_reg()` (in the nle repo), and a zero-box 8-parameter scaffold card, which is not committed. For the probe in §1.3 I made a throw-away scaffold from the committed `ddm_cap256w_c4` card by relabelling `sv, SZ, st0` as `zerobox` with the zero-box constants and bounds; it is in this session's scratch space only and proves only that the plumbing works. The right fix is a card kind (say `flow_scaled`) that the R loader understands, so the scaffold disappears.
4. **Tail mass.** With `range_max = 2` on log u and identity tails, the flow puts exactly $1 - \Phi(2) = 2.275$ % of its mass above $u = e^2$ for every θ (the sum of the two defective CDFs at `range_max` is 0.97725 for five random θ; quadrature over the range gives 0.9757). This is a property of the construction, θ-independent, so it is a constant in the log-likelihood and cannot bias a posterior, and the SBC agrees. But `nn_total_mass()` on this card will read 0.977, and the user documentation (and the card spec, §7.1) should state the tail mass of a card rather than let a user discover it. AH may already know this; I did not find it written down.
5. **The two score-bias regimes not covered by the committed material**: the screen scripts under `sbc/scaled_screen/` reference cards under `port/export/` on the laptop; only the three selected cards are committed, so the selection cannot be re-run from the repository (by design: `RECIPE.md:17-18`, regenerate rather than deposit).

---

3. Worked example B: the RDM

The race model is the generality case, not the speed case (`METHODS.md:73-78, 106`): the flow models one accumulator's first-passage density f and survivor S, and the race likelihood is assembled as $L(rt, R \mid \theta) = f_R(rt) \cdot \Pi_{j \neq R} S_j(rt)$ (`METHODS.md:127-135`; `port/R/flow_race.R:9-17`). No classifier.

3.1 The simulator, box and the three RDM flows that exist

	`rdm_small` (committed)	**A = 0 flow `rdm_ss_base_s801_e271`**	**Malte (conv**
context	`v, B, t0, s, A`, all `log`, standardised by the training moments	same, with `A` as `zerobox`, c_A = 0.05 (`RDM_A0_BOX_PLAN.md:25-30`)	`v, s, tau, s` **scale**, (log(x scale

	rdm_small (committed)	**A = 0 flow rdm_ss_base_s801_e271**	**Malte (conv**
			(MALTE 45)
box (natural)	lower (1e-3, 0.1, 0.05, 0.1, 1e-4), upper (5, 3, 1, 2, 2) (export_flow.py:74-94)	A ∈ [0, 2], continuous to 0; three-part mixture 70 % uniform / 15 % asinh-uniform / 15 % exactly 0 (RDM_A0_BOX_PLAN.md:32-46)	v [0, 8 (CRDM 0.5], s
data	16e6 rows, one trial per draw, analytic ll stored, seed 3876 (simulation/sim_rdm.R)	24e6 rows, seed 4876, simulated on tux26 in under a minute (RDM_A0_BOX_PLAN.md:161-167)	online step, r (MALTE
architecture	[64, 32], 64 bins, range ±5 (METHODS.md:138-143; card rdm_rdm_small_e195_weights.json)	[256, 128], 64 bins (RDM_A0_BOX_PLAN.md:50-52)	[128, 1 contex "t0" (
selection	checkpoint e195 of 30, by score bias (METHODS.md:219-220)	e271 of the seed study's baseline arm (120 screened checkpoints, MALTE_STAGES.md "M4 part E")	single
SBC	N = 100 and 400, 500 reps, all parameters inside the gate, ranks uniform (METHODS.md:224-233)	Gadi, 2026-10-06, final sampler: A = 0 cell N 400 (250 reps) and standard cells N 100/400 (500 reps), network and control,	std bia
weights in the repo	**yes**: handover/nle-methods/artefacts/rdm_rdm_small_e195_weights.json and rdm_small.rds; golden rdm_rdm_small_e195_golden.csv in the EMC2 test tree	**no** (the Gadi bundles hold cell.rds, run_cell.R, session.txt; the card lived in port/export, untracked)	**no** tra rando golden malte/

Which of these is "the RDM" for the package is a decision for AH (§11.3): rdm_small is the committed, documented artefact; the A = 0 flow is the one the comparison with Malte and the seed study are built on and the one a user fitting a Wald race (A = 0) needs.

3.2 Screening, the seed study, and the EMC2 side

Score bias for a race model: handover/nle-methods/evaluation/score_bias_race.R (H = 0.01, the trapezoid measure, t0 by the restricted-region estimator; analytic control ~1e-4, METHODS.md:157-182). The seed study (handover/MALTE_STAGES.md M7a/M7b, five arms × four seeds: baseline, affine, decision time, both, Malte's recipe; sbc/seed_study_launch_tux.sh, sbc/seed_study_screen.sh) has its baseline arm screened (sbc/seed_study/ss/summary_base.csv); the full screen and its reading are the handed-over stage at the bottom of MALTE_STAGES.md and are not in the log as of this commit. So whether --affine and --decision-time are part of "the recipe" or experiments is still open, and the package should carry them as options, not defaults.

EMC2 side: register_nn_model(card, twin = RDM); for a card without A, pars = c("v","s","B") with exceptions = c(A = 0) (sbc/gadi/cells/build_rdm_cell.R:30-45); Malte's cards register with hybrid = "amp" so that the unpulsed accumulator is scored by the closed-form Wald (nn_register.R:665-682). The compiled race evaluator takes the input_scaling and affine fields (src/flow_race.cpp:153-200).

3.3 What the package would need to reproduce it

A spec with: context `v, B, t0, s, A` (or `v, s, b` with `A` fixed at 0), transforms log/zerobox, the box, the A mixture; a single-accumulator simulator wrapper on EMC2's `rWald` writing `rt, ll, param_id` and the context; `response = "race"` (no classifier; f and S from one network); optional `pre = "t0"` and `affine`. The Malte path is a converter (`flow/scripts/malte_to_card.py`) rather than training: it reads an Orbax checkpoint plus a sidecar and writes a card with `context_scale = "natural"`, `input_scaling`, `affine`, `affine_scale`, `pre` (`:124-151`).

---

4. What the two share, where they differ, and the interface that follows

4.1 Shared (the package core)

- A conditional rational-quadratic spline flow over a log time, one spline, standard-normal base, identity tails, GELU-tanh MLP conditioner; the float64 NumPy port that generates golden vectors; the port-vs-JAX gate.
- The simulate → train (one run, large batch, many checkpoints) → export → screen on score bias → select → SBC-with-matched-control recipe (`RECIPE.md`).
- The card: `context_names`, `context_transforms` (+ `zerobox_c`), `bounds_natural`/`bounds_sampled`, `scaler` or `input_scaling`, `mlp`/`flow_mlp`, `spline`, `ll_floor_log`, `pre`.
- The validation kit in EMC2, which is model-agnostic given a registered model with an analytic twin.

4.2 Different (the model-specification interface)

axis	DDMnle	RDM	what the spec must carry
response structure	two responses; P(R	θ) from a separate classifier on an exact target; the flow sees R as an input	race; one network per accumulator, f and S; no classifier
exact zeros	η = 0, SZ = 0 (asinh encoding); st0 = 0 is the plain density	A = 0 (asinh); Malte: amp = 0 is the Wald (hybrid)	per-input `zerobox` with its constant; stratified regimes
closed-form transforms	a, s rescaling; t0 shift; st0 by CDF difference	t0 shift (decision time)	`pre` (shift) and a `scaling` rule (new: §7.1)
analytic control	yes (WienR / `dDDM`)	yes (`dRDM`, Wald); CRDM: Volterra reference	the twin's name, or a reference `dfun`
input scale	sampled (EMC2) with a `scaler`	sampled, or natural with `input_scaling` (Malte)	both; the loader has both
spline range	[−12, 2] (set from pilot data)	[−5, 5]	a spec field, not a constant

4.3 The hard-coded pieces this exposes

Python (`handover/nle-methods/flow/`): `CONTEXT*` literals (`train_ddm.py:43-62`, `export_flow_ddm.py:45-61`, `retrain_classifier.py:31-56`), `CONTEXT`/`MODEL_REGISTRY` (`train_race.py:71-74`, `export_flow.py:74-94`); `TRANSFORMS`/`BOUNDS_NAT`/`ZEROBOX_C`/`ZEROBOX_BOUNDS_NAT` (`export_flow_ddm.py:74-75, 111-119`; `data.py:87`; `train_race.py:102`; `export_flow.py:69`); the spline range fixed at ±5 in the race exporter (`export_flow.py:372-375`) but read from `variant_config.json` by the DDM one; column names `rt, R, ll, param_id, p_R2, d`; binary R with the P(R = 2) logit (`config.py:269-276`); the edge loss's DDM column indices (`training.py:65-66`); `SHIFT = "t0"` (`train_race.py:79`) and `pre = "t0"` always in `malte_to_card.py:129`; `results/model_states/<tag>_budget/{DDM_flow, DDM_classifier,RDM,LNR}` and `port/export/` paths; `REPO_ROOT = parents[2]`; the classifier placeholder copied from `--classifier-from large` (`train_ddm.py:219-230`); golden seeds and rt grids.

R: `DDM_SC_INPUTS`, `DDM_SC_C` and the eight-column `sc_pars` (`ddm_scaled.R:25-26, 133`); `SC_*` constants and regimes (`scaled.R:13-25`); `scaled_cell.R`'s scaffold and regime table (`:27-31`); the eight-case drivers.

EMC2: joint kinds are two-response (`model_NN.h:165-169`, `nn_register.R:1108-1109`); twin table DDM, RDM, LNR, LBA, CRDM (`nn_register.R:59`); zero-box names `sv, SZ, st0` and the `st0zero` rule (`:601-619`); the support rule looks up `st0` and `t0` by name (`flow_ddm.cpp:281-298`); hybrid Wald map `c(v, b → B, s)` (`:674`); the `triple` encoding's column positions (`flow_ddm.cpp:68-80`); the scaled patch's exactly-four-inputs and eight DDM columns.

4.4 Details the documentation needs that are not written down (ask AH)

1. Which RDM artefact is the worked example (§3.1), and whether the A = 0 flow's card should be committed.
2. The SZ convention at every interface: the network's SZ is the *relative* width (SZ/m, `ddm_scaled.R:30-35`); EMC2's `DDM()` passes `sw = 2 SZ min(Z, 1 − Z)` after `Ttransform`; the registered scaffold passes a proportion (`scaled_cell.R:53-57`). This is in comments only.
3. The s = 1 rule in every RDM control cell (`build_rdm_cell.R:12-13`, "pWald overflows for v b/s² > 354") and the ±4 sd prior-inside-box rule (`nn_validate.R:94`, `sbc_specs.R:6-9`): are these package defaults or cell conventions?
4. Whether `--affine` and `--decision-time` are recipe or experiment (the seed-study reading is pending).
5. The tail-mass convention (§2.4.4) and what `nn_total_mass()` should expect for a card.
6. The t0 correction's status for the old DDM artefacts (superseded by DDMnle for case 8; `OPEN_ITEMS.md:31-35`): the package should not carry it, but the documentation of `cap256w_c4` must say what applies.
7. `rt_resolution ≤ 1/1000` for t0-sensitive work (`METHODS.md:714-715`): a package default or a vignette note?
8. Where the DDMnle checkpoints are (tux26, `~/emc2nn/nle/handover/nle-methods/results/model_states/sc_s1106_budget` by the pipeline's layout) so the golden vectors can be regenerated (§2.4.2).

---

5. Scope and audience

who	what they cannot do now	what they would do with this	needs
us (Julian, Malte, AH)	reproduce DDMnle or the RDM flow without the laptop's `port/export`, tux26 checkpoints and a session's memory of which flag; verify a card without the nle repo	re-run either example from a spec on a fresh machine; add the next model (CRDM-style, LBA variants) by writing a spec; ship cards with golden vectors	the Python package + the EMC2 pieces in §0
EMC2 users with an intractable model	train anything; the only route is to copy `flow/` and edit six files; nothing documents the traps in one place	write a simulator and a spec, run one command on a workstation or cluster, get a card, register it, run the kit before trusting it	the same, plus the installation docs of §9
HSSM / LANfactory users wanting EMC2	nothing they cannot do today in principle: `onnx_to_card.py` exists and the LAN registers with a hand-written twin (`sbc/lan_case5/lan_model.R`); but it supports tanh MLPs with a rank-1 input only, one preset, and the user must know the parameter mapping and must run an SBC cell before inference (`EMC2_IMPLEMENTATION_BRIEF.md:175-184`; the one LAN scored carries g_a = −0.122, `METHODS.md:434`)	a converter that reads the HSSM model config for the mapping and box, accepts batched graphs and ReLU/other activations, and refuses what it cannot represent	the converter moved into the Python package and extended; a twin-writing helper in EMC2
HSSM users wanting DDMnle	nothing: the DDMnle ONNX graph does not exist (§2.4.1)	load one ONNX file with the LAN-shaped signature	the ONNX exporter for the scaled kind, and an HSSM model config (outside our control)

Overlap. EMC2 `dev-nle` already does the import, validation and fitting half; this plan does not move any of it out. HSSM/LANfactory train LANs (per-trial MLPs, Python, ONNX out) and have nothing for flows with amortised conditioning or for calibration beyond parameter recovery; our converter consumes their output. BayesFlow and sbi do amortised posterior/likelihood estimation in Python with their own samplers, not EMC2's; the only overlap is the word "flow". Nothing in the field ships the score-bias/posterior-shift/SBC-with-control kit, which is EMC2's.

Where each piece lives

piece	lives in	why
model spec, simulator glue (parquet writer), training, checkpoint export with golden vectors, float64 gate	Python `nleflow`	JAX; the port that makes the golden vectors is NumPy
ONNX export (both signatures, all kinds), `onnx_check`	Python `nleflow`	onnx helper API
ONNX → card converter	Python `nleflow` (moved from `EMC2/inst/scripts`)	it already requires Python; EMC2 keeps a one-line pointer
card format spec and version, loader, `register_nn_model`,	EMC2	they are there, tested, and must not be duplicated

piece	lives in	why
compiled evaluators, the kit, `nn_golden_check`		
R reference evaluators (`flow_race.R`, `flow_ddm.R`, `ddm_scaled.R`)	nle repo now; EMC2 `tests/testthat` fixtures (two of three already are, `test-nn-port.R`)	the readable spec of the C++
the two worked examples (specs, small smoke data, cards, golden, SBC results)	`nleflow` examples + the nle repo's `handover/nle-methods/`	vignettes and end-to-end tests
Malte converter (`malte_to_card.py`)	Python `nleflow` as an optional "converters" module	its environment (jax ≥ 0.9) conflicts with ours; keep it isolated

---

6. Architecture

option	what it is	for	against
A. R package wrapping Python (reticulate)	`nleFlow::train(spec)` calls JAX through reticulate; export, convert and validate in R	one language for the user; `design()`-style spec in R	reticulate + a pinned JAX env + optional CUDA in an R package: not CRAN-able, fragile across JAX releases (0.5.3 here, 0.10.2 for Malte, both in this repo); every evaluator would be re-implemented in R or copied from EMC2 (drift); the dedicated-library trap gains a Python twin
B. Python package + R front end	`nleflow` (Python) does spec, train, export, ONNX, convert; EMC2 does cards and validation; a thin R helper (in EMC2 or `nleflow`'s `R/` dir) writes the spec JSON and the parquet from `make_data`	each half in its natural language; EMC2 stays the single R evaluator; the R helper is ~200 lines	two installs (R + Python); Python needed to train, which is true of every option
C. R-only tools + documented recipe	an R package for export/convert/validate; training stays "run these scripts"	smallest	duplicates `dev-nle`; converting ONNX needs Python anyway; the scripts stay unusable for a new model

Recommendation: B, with the R front end inside EMC2 rather than a separate package. The R parts that must be R are: the simulator glue (`make_data` → parquet; this is the one place the user's EMC2 model function is needed), the card loader and compiled evaluators, the kit, and the golden check. All of those exist in EMC2 except the parquet writer (today `simulation/sim_functions.R`'s `simulate_data` in the nle repo, with `arrow`) and the golden check.

Dependency and installation burden. EMC2 `dev-nle` adds `jsonlite` (Suggests, `DESCRIPTION:47`) and `loo` (Imports) over the 3.5.0 release, and pins `WienR (>= 0.3-17)`; no reticulate, no onnx. It needs a C++17 compiler, as EMC2 always has. The Python side is a pinned `uv` environment (11 s here). GPU is optional (`jax[cuda12]` extra); DDMnle trained on

CPU. CRAN: the EMC2 NN code is CRAN-compatible as far as dependencies go; whether `dev-nle` goes into the next release is AH's and Niek's call and outside this plan. The Python package goes to PyPI or stays a GitHub install; either way with a lock.

The dedicated-library trap stays whatever is built, because `dev-nle` and the release share a version prefix (3.4.1.9001 vs 3.4.1 in the dev-nn era; the ampl-psych checkout here is 3.5.0). The docs and the verification command check `find.package("EMC2")`, never the version (§9.3).

---

7. Interfaces

7.1 The card format: version it, write it down

Today the card is whatever `export_flow*.py` writes and `nn_normalise_card()` / the C++ loaders accept (`nn_register.R:155-180`; `flow_ddm.cpp:256-309`; `flow_race.cpp:153-200`; `mlp_lik.cpp:90-141`; `nle_flow.h:66-97, 170-181`). There is no version field and no document; `man/register_nn_model.Rd` describes the fields in prose. Proposed `card_version = 1`, with the spec living in EMC2 (`vignettes/` or `inst/doc`) and mirrored in `nleflow`:

group	fields	read by
identity	`card_version`, `kind` (flow_joint / flow_race / mlp_joint / regression_joint / **flow_scaled**), `model`, `label`, `source`, `sha256` of the weights	R
inputs	`context_names`, `context_transforms` (identity, log, probit, zerobox), `zerobox_c`, `context_scale` (sampled/natural), `scaler` or `input_scaling`, `bounds_natural`, `bounds_sampled`	R and C++
structure	`pre` (shift parameter), `hybrid`, `response_coding` / `response_values`, `input_layout` (MLP kinds), **`scaling`** (the DDMnle rule, as a structured field: `{u: "(rt - t0) * (s/a)^2", inputs: {kappa: "v*a/s^2", eta: "sv*a/s^2"}, smear: "st0"}` or an enumerated kind)	R and C++
network	`mlp/flow_mlp/classifier_mlp` (`activation`, `use_norm`, `layers`, `norms`), `affine`, `affine_scale`	C++
spline	`num_bins`, `num_splines = 1`, `range_min`, `range_max`, `min_bin_size`, `min_knot_slope`, `boundary_slopes = identity`, `output_transform = exp`, `base_distribution`	C++
floors and mass	`ll_floor_log`, **`tail_mass`** ($1 - \Phi(\text{range_max}) + \Phi(\text{range_min})$, §2.4.4)	R
provenance	`budget`, `checkpoint_step*`, `exported`, `exporter_version`,	R, not required

group	fields	read by
	`golden` (path or embedded hash of the golden file), `validation` (score-bias, posterior-shift and SBC records with dates)	

Rules: unknown fields are carried along; a card without `card_version` is version 0 and loads as today; a loader refuses a version it does not know. `register_nn_model()` already refuses mis-ordered contexts and missing transforms; it should additionally refuse a card whose golden file is present and fails (`nn_golden_check`).

7.2 The model specification a user supplies (Python; an R helper writes the same JSON)

```
        model: RDM_A0                 # a name; also the twin's name when it is an EMC2
model
        response: race                # race | choice
        context: [v, B, t0, s, A]     # in network order; asserted everywhere
        transforms: {v: log, B: log, t0: log, s: log, A: zerobox}
        zerobox_c: {A: 0.05}
        box_natural: {v: [1e-3, 5], B: [0.1, 3], t0: [0.05, 1], s: [0.1, 2], A: [0, 2]}
        sampling: {A: {uniform: 0.70, asinh_uniform: 0.15, zero: 0.15}}   # strata; the rest
uniform in the transform
        pre: t0                       # optional: the flow models rt - t0
        scaling: null                 # DDMnle: {kind: ddm_unit}
        spline: {bins: 64, range: [-5, 5]}
        conditioner: [256, 128]
        classifier: null              # choice models: {target: p_R2, widths: [256, 256,
256]}
        training: {rows: 24e6, trials_per_draw: 1, batch: 1024, epochs: 300, keep: 30,
peak_lr: 1e-4, seeds: [801, 802, 803, 804]}
        simulator: Rscript simulate.R  # anything that writes the parquet columns below
```

Parquet contract (today's loader, `data.py:107-160`): one row per trial, columns `param_id`, the context on the sampled scale (−Inf at exact zeros), `rt` (or the pre-shifted time), `R` for choice models, `ll` (the analytic log density where it exists; the loader reads it even for plain MLE, so make it optional in the package), and for choice models a second file with the context plus `p_R2`.

7.3 ONNX signatures

Two, as now (`export_onnx_ddm.py:9-27, 92-107`): **trial-wise** `[N, k+2]` (context on the sampled scale, rt, choice ∈ {−1, +1}) → `[N]` log p, the LAN-shaped contract for HSSM; and **shared-theta** `theta[k], rt[N], choice[N]`, 25x faster because the conditioner runs twice and the classifier once (`EMC2_INTEGRATION.md:164-179`). Both float64 (float32 rejected at 1.9e-4, `:185-186`); opset 18, IR 10 for onnxruntime ≤ 1.30 (`:264-266`; 1.31.0 here loads them). For `flow_scaled` the graph takes the natural DDM parameters and does κ, η, c, the shift and the CDF difference inside (`REPORT_RUSS_JULIAN.md:160-165`), with probit and log left to the caller. For race cards an ONNX export is possible per accumulator (f and S), but no consumer exists; defer.

7.4 The ONNX → card converter

Supported today (`onnx_to_card.py:69-147`): one non-initializer input, `MatMul`+`Add` or `Gemm`, `Tanh`, `Identity`, one linear output; rank-1 input assumed by `--verify` (`:263-266`); one preset. To add, in order of value: batched `[N, k]` inputs (HSSM batches by converting the graph; `EMC2_IMPLEMENTATION_BRIEF.md:123-131`), `Relu`/`Gelu` activations (the evaluator `load_mlp` accepts `gelu_tanh` and `tanh` only, `nle_flow.h:66-97`; each new activation is a C++ change too), a preset per published LAN with its parameter mapping and box read from the HSSM model config, and a refusal message that names the first unsupported node. The converter must keep writing `calibration: "unknown; needs its own SBC cell before inference"` (as the LAN card does: `sbc/lan_case5/lan_ddm_uniform_st.rds`).

7.5 Validation built into export, so that a card cannot be exported without passing

gate	where it runs	exists
port vs JAX float64 ≤ 1e-8 (float32 gap reported)	exporter	yes for the DDM (`export_flow_ddm.py:429-435`); the race exporter gates against float32 only (`export_flow.py:471-507`)
golden vectors written with the card; `nn_golden_check(card)` in EMC2 reproduces them to 1e-10	exporter + EMC2	golden yes; the R check only as test code
context order and transforms asserted on load	EMC2	yes
total mass = 1 − tail mass	EMC2 kit	`nn_total_mass` exists; the expectation needs the card field
score bias with its analytic control, control < 1e-3	EMC2 kit / `score_bias_*.R`	yes
posterior-shift screen	EMC2 kit	yes
one SBC cell with a matched control; result recorded in the card's `validation` block	EMC2 kit	the cell yes; the record no

The export command should refuse to write a card whose port gate fails, and `register_nn_model()` should warn when a card has no `validation.sbc` entry (that is the honest default: "loads, not yet fit to infer with").

---

8. Reusable now, needs generalising, missing

Reusable as is (would be moved, not rewritten): the NumPy float64 port and gate (`export_flow.py:116-254`, `export_flow_ddm.py:172-204, 380-451`); `export_onnx_ddm.py` (graph construction); `onnx_check.py`; `onnx_to_card.py`; `malte_to_card.py`; `port_x64_check.py`; the loader `PreloadedBatches` and the training loop (`training.py:470-596`), minus the DDM-specific losses; `checkpoints.py`; the R reference evaluators (`port/R/*.R`); the score-bias and posterior-shift estimators (already in EMC2 as `nn_score_bias`, `nn_posterior_shift`); `simulate_data` (`simulation/sim_functions.R`) as the parquet writer; EMC2's whole NN layer.

Needs generalising: everything in §4.3. Concretely: one `train` command that takes a spec (replacing `train_ddm.py`/`train_race.py`); one `export` command (replacing `export_flow.py`/`export_flow_ddm.py`), with the float64 gate for every kind and the spline range from the spec; the classifier step without the copy-a-placeholder-checkpoint dance (`train_ddm.py:219-230`); `apply_context_encoding` keyed by the spec's transforms rather than by parameter name (`data.py:90-105`); the race exporter's ±5 refusal; `malte_to_card.py`'s `pre = "t0"` always; EMC2's `flow_scaled` kind in R; EMC2's two-response assumption left as a documented limit for now.

Missing: the DDMnle ONNX graph (§2.4.1); DDMnle golden vectors for the committed card (§2.4.2); `nn_golden_check`; the card version and spec; `tail_mass`; a verification command (§9.4); the R < 4.5 hash fix (`nle_file_hash` writes md5 into `MANIFEST` or refuses R < 4.5 with a clear message); `onnx`/`onnxruntime` in the Python lock; a README and `requirements.txt` in the handover `flow/` (`pyproject.toml` declares `readme = "README.md"`, which is absent there, so an editable install of the handover copy would fail); the training loop's `avg_loss` bug for sets under 10 batches (`training.py:432-436, 463-467`); the end-to-end smoke tests of the two examples on 1 % data (the pattern exists: `sbc/seed_study/a0smoke/`, `m7asmoke/`, the 2-epoch smoke in `SCALED_DDM_PLAN.md:115-122`).

---

9. Installation documentation (scoped; what I tested)

9.1 Inventory per function group

function group	needs	version actually used (evidence)	not recorded
use a card in EMC2 (fit, kit)	R; EMC2 `dev-nle` built from source (C++17 compiler, OpenMP optional); Rcpp, RcppArmadillo, WienR ≥ 0.3-17, jsonlite, and EMC2's Imports	laptop R 4.6.0, macOS 14.5, reference BLAS, WienR 0.3-17, Rcpp 1.1.2, jsonlite 2.0.0 (`sbc/gadi/cells/*/bundle/session.txt`); Gadi `module load R/4.5.0`, gcc 12.2.0 + intel-compiler 2021.10.0, MKL, `-fp-model=precise` (`sbc/nci/*.pbs:23-25`, `MALTE_STAGES.md:1110`); here R 4.3.3	tux26 and AWS: no R version anywhere; macOS compiler/Xcode; **R < 4.5 breaks shipped artefacts** (§1.3)
train	Python 3.11–3.12, `uv`, the lock (jax 0.5.3 etc.); `arrow` in R for the simulator (plus `randtoolbox` for Sobol in the scaled simulator); EMC2 for `make_data`	tux26 python 3.12 (`MALTE_STAGES.md:338`); laptop interpreter version unknown; CPU only for every run that shipped	GPU never used; the laptop's Python patch version
export to EMC2 card	the same Python env (NumPy port)	as above	
export to ONNX	the Python env + `onnx` (+ `onnxruntime` to check)	"onnxruntime 1.30, onnx 1.23" in a comment only (`export_onnx_ddm.py:264`); here onnx 1.23.2, ort 1.31.0	which machine; not in any lock
convert ONNX → card	Python + `onnx` + numpy (+ onnxruntime for `--verify`)	as above	
SBC and score-bias checks	EMC2 `dev-nle`, WienR; cores	laptop 8 cores (policy); AWS c7a.32xlarge 128 cores, 6.45 h, ≈ $45 for 18 cells (`SCALED_DDM_PLAN.md:196-208`); Gadi 48 cpus per cell job	
Malte's converter	its own env: Python 3.11, jax 0.10.2, flax 0.12.8,	recorded	

function group	needs	version actually used (evidence)	not recorded
	malte/requirements.lock		

9.2 Per-platform steps (each with its one-line check)

Order matters: R and its packages, then EMC2 into a dedicated library, then the card check, then Python only if training or ONNX is needed.

macOS (Apple Silicon; what the project ran on) 1. R ≥ 4.5 from CRAN's installer. Check: `R --version | head -1` shows ≥ 4.5 (4.3/4.4 hit the hash trap). 2. Xcode command-line tools and Homebrew `libomp` (EMC2's `configure:104-151` links Accelerate and uses libomp for OpenMP). Check: `clang++ --version` prints; `R CMD config CXX17` prints a compiler. 3. `install.packages(c("Rcpp","RcppArmadillo","WienR","jsonlite","arrow","randtoolbox", <EMC2 Imports>))`. Check: `Rscript -e 'stopifnot(packageVersion("WienR") >= "0.3.17")'`. 4. EMC2 `dev-nle`: `R CMD INSTALL --preclean --library=~/R/nle-lib <EMC2 source>` (the tarball, or `git clone -b dev-nle`). Check: `Rscript -e '.libPaths(c("~/R/nle-lib", .libPaths())); library(EMC2); cat(find.package("EMC2"), "\n")'` prints a path under `~/R/nle-lib`. 5. The verification command (§9.4). 6. (training/ONNX) `uv venv --python 3.11 && uv sync --frozen` in the Python package; `uv pip install onnx onnxruntime` until they are in the lock. Check: `python -c "import jax, flax, onnx, onnxruntime; print(jax.__version__)"` prints 0.5.3. Set `JAX_ENABLE_X64=1` only for the export gate (the exporter sets it itself, `export_flow_ddm.py:259-260`); training runs float32.

Not tested on macOS in this stage (no macOS here); the steps are what the repo did, with the checks added.

Linux workstation (tux26-like) 1. R ≥ 4.5 (CRAN's Ubuntu repository or a module). Check as above. On a shared machine with another EMC2 in the default library, `R_PROFILE_USER=/dev/null` for every run (as `sbc/aws/scaled_sbc/bundle/smoke.sh` does). 2. `g++` ≥ 11 (`CXX_STD = CXX17`), `libopenblas` optional. Check: `R CMD config CXX17`. 3. R packages as on macOS (Ubuntu: `apt-get install r-cran-{rcpp,rcpparmadillo,...}` is faster; WienR is not packaged). Check as above. 4. EMC2 `dev-nle` into `~/R/nle-lib` with `MAKEFLAGS=-j$(nproc)`. Check: `find.package`. 5. Verification command. 6. Python by `uv` as above; `OMP_NUM_THREADS` per run (the pipeline used 4 per seed, `scaled_pipeline_tux.sh:7`).

Cloud instance (this one; AWS recorded in `sbc/aws/scaled_sbc/bundle/setup.sh`) 1. `apt-get install r-base-core r-base-dev` (45 s here; R 4.3.3 on Ubuntu 24.04, so step 4's check matters and the shipped-artefact trap applies: use JSON cards, or R ≥ 4.5 from CRAN's repository). 2. `apt-get install r-cran-<each>` for the Imports (2 min 16 s), WienR from source (18 s), `loo`, `msm` as needed. Check: `Rscript -e 'for (p in c("WienR","jsonlite","Rcpp","RcppArmadillo","loo")) stopifnot(requireNamespace(p))'`. 3. `tar -xzf emc2_142bd497.tar.gz; MAKEFLAGS=-j$(nproc) R CMD INSTALL --preclean --library=/opt/emc2/lib <src>` (78 s here with 4 cores; `setup.sh:11-19` writes `EMC2_SHA.txt` beside the library, which is a good convention). Check: `find.package`. 4. Verification command. 5. `uv` + the lock (11 s here). Check as above.

Traps to print in the docs (all hit in this project): the dual install and `find.package` (`EMC2_INTEGRATION.md:8-24`); `R_PROFILE_USER=/dev/null` on shared machines; a card's absolute path is baked into a saved registration/bundle (`nn_register.R:709`, `nn_validate.R:1014-1015`; the Gadi bundles once "never fitted a replicate" for this, `MALTE_STAGES.md:1112-1114`); the float64 export gate for scaled cards; EMC2 ships its own compiled copy of the port, so

edits to `port/src` prove nothing about EMC2; `prior()` positional means; `mapped_pars()` asserted for every parameter not sampled; R < 4.5 hash trap; `onnx`/`onnxruntime` not in the lock.

9.3 The single verification command

Proposed `nleflow-verify` (an R script shipped with EMC2 as `inst/scripts/nle_verify.R`, or `Rscript -e 'EMC2::nn_verify()'`), installing nothing. It: 1. prints `R.version.string`, `find.package("EMC2")`, `packageVersion("EMC2")` and the library's `EMC2_SHA.txt`, and stops if R < 4.5 with the hash-trap message; 2. loads each committed card (DDMnle `ddm_sc_s1106_e245`, `rdm_small`, `ddm_cap256w_c4`, the LAN card) through `register_nn_model()`, asserting context names and transforms; 3. evaluates the compiled likelihood at the golden rows and compares with the golden file to 1e-10 (`nn_golden_check`); for the LAN, with the onnxruntime golden CSV at 1e-4 (float32 source); 4. for DDMnle, also runs the oracle test (`port/tests/test_ddm_scaled.R`, 7 s) and the compiled-vs-R agreement on 200 draws (`scaled_cell.R`'s check, 7e-13); 5. prints one line per card, `PASS max|diff| 6.9e-13`, and exits non-zero on any failure.

What it can check today: `rdm_small` and `cap256w_c4` (golden in EMC2's test tree); the oracle test; the LAN card. What it cannot until §2.4.2 is done: DDMnle against golden vectors (none exist for the committed card), and the compiled DDMnle path through `register_nn_model()` (needs the `flow_scaled` kind, or the scaffold).

9.4 Scripted or documented?

Documented steps with one check per step, plus a container, plus the verification command; not a setup function in the package. A setup function that installs R packages and builds EMC2 into a chosen library is thirty lines and I would not trust it on a machine with another EMC2 on the path; `sbc/aws/scaled_sbc/bundle/setup.sh` is already that script for Ubuntu and should be kept as the cloud recipe (it is 20 lines and worked). A `Dockerfile` (Ubuntu 24.04, R from CRAN's repository so that R ≥ 4.5, the apt packages, WienR, EMC2 from the tarball or the branch, `uv` with the lock) reproduces what I ran here in about 5 minutes and gives a cluster a Singularity image. Effort: 2 days for the docs with checks, 1 day for the container, 2 days for the verification command (half of it the golden check in EMC2). Risk: low; the one real risk is the R-version trap, which the command detects.

9.5 What I ran and what I only wrote

Ran (this instance): §1.3's table in full; the cloud steps 1–5 above except the verification command, which does not exist yet (its pieces ran separately: oracle test, ONNX check, converter, EMC2's nn tests, the compiled-path probe). Wrote only: the macOS and Linux step lists (they are the project's recorded practice plus checks), the container, the verification command.

---

10. Effort, risk, maintenance

10.1 Work breakdown (days, one person who knows the code; Claude-assisted as the project has been)

item	days	who
M0 first milestone (§11.1): card spec v1 + version field; `flow_scaled` kind in R (wire the patch); `nn_golden_check`; DDMnle golden vectors regenerated (tux26 run, minutes); R < 4.5 fix; `nle_verify`	5–7	EMC2 session (Fable/Opus: trap-dense) + one tux26 export run
Python package skeleton: spec schema, loader keyed by spec, one `train`, one `export` with the float64 gate for all kinds, lock with onnx/onnxruntime, README	8–10	Sonnet-grade against the two examples' smoke tests
the two examples as specs + 1 % smoke tests + vignettes (DDMnle, RDM A = 0 or `rdm_small`)	3–4	
ONNX: generic exporter over card kinds; the DDMnle graph with CDF, preprocessing and st0 branch; acceptance vs compiled path on 200 draws × 6 models	4–5	needs the compiled path (M0)
converter: move into the package; batched inputs; activations; presets from HSSM configs; refusal messages	2–3	each new activation is also a C++ line in `load_mlp`
EMC2 rebase of `dev-nle` onto `dev` (the new sampler is already on both sides, differently) and re-running the usability gate (`EMC2_IMPLEMENTATION_BRIEF.md:301-351`, 25 min at 8 cores)	2–3	EMC2 session
installation docs, container, CI (GitHub Actions on Ubuntu: build EMC2 from the branch, run `nle_verify`)	3–4	
total	**27–36**	

Compute: no training is needed for any of it except the smoke tests (minutes). The DDMnle ONNX acceptance test is seconds. Regenerating golden vectors is one export on tux26 (ask first, as always).

10.2 Risks

risk	where it bites	mitigation
format drift between the exporter and EMC2's loader	the only interface that matters; today there is no version	version the card; a loader that refuses unknown versions; `nle_verify` in CI on both repos; EMC2's `test-nn-*` already pin the three shipped cards
EMC2 `dev-nle` vs `dev`	`dev-nle` is behind `dev` by the sampler commits of 6–9 Oct and ahead by the NN layer; 142bd497 and the scaled patch are local	rebase before M0; push the patch as a branch so it is not a file in another repo

risk	where it bites	mitigation
ONNX operator coverage	the DDMnle graph needs `Erf` (opset ≥ 9, fine) and the CDF-difference branch; `Where`-heavy graphs are slow in onnxruntime for the trial-wise signature	keep both signatures; accept the trial-wise one as the HSSM contract and the shared one as the fast path
JAX / flax / orbax churn	0.5.3 vs Malte's 0.10.2 already needed two environments; orbax restore formats changed between them (`MALTE_STAGES.md` M4 part A)	pin by lock; the exported card is the durable artefact, the checkpoint is not; keep the NumPy port as the thing that reads weights
R version	R < 4.5 silently cannot load shipped artefacts	fix `nle_file_hash`; `nle_verify` prints the R version first
two copies of the Python code (`flow/` and `handover/nle-methods/flow/`)	they have already diverged	the package replaces both; the handover copy becomes the examples
support burden	a likelihood that loads silently and fits silently (`EMC2_IMPLEMENTATION_BRIEF.md:244-247`)	the kit is the price of admission: `register_nn_model()` warns without a `validation.sbc` record

10.3 Maintenance

The Python package: Julian (author of `flow/`) with AH; it changes when a new model family needs a new structural option, which is rare. The EMC2 NN layer: whoever maintains `dev-nle` now (the EMC2 sessions under AH), inside EMC2's test suite, which is the right place because EMC2's samplers are what it is tested against. Malte's converter: Malte. If nobody will own the Python side, the honest alternative is option C (§6) with the scripts frozen at a tag and documented as "edit these six places": cheaper, and it is what exists now.

---

11. Recommendation, first milestone, decisions

11.1 The smallest useful first milestone (M0, about a week)

It is independent of the Python-package decision and it removes the three gaps found in this stage:

1. **Card spec v1**, written in EMC2 (`vignettes/card-format.Rmd` or `inst/doc/CARD.md`), with `card_version`, `tail_mass`, a structured `scaling`, and the `validation` block; `nn_normalise_card()` reads the version.
2. **`flow_scaled` in R**: `register_nn_model()` recognises a DDMnle card (four inputs, `scaling` present), builds the native spec the patch expects, and fills `fixed`/`refuse` from `DDM()`; `ddm_sc_native_reg()` and the scaffold go away; `DDMsc()`'s R path becomes the R-path twin of the compiled kind (like the other kinds). Acceptance: compiled vs `ddm_sc_density` ≤ 1e-11 on 200 draws × 6 models (the existing check in `scaled_cell.R`).
3. **`nn_golden_check(card, golden)`** in EMC2, used by the tests and by `nle_verify`.
4. **DDMnle golden vectors** for `ddm_sc_s1106_e245` (one tux26 export; AH to approve), committed beside the card, plus the two runners-up if they are kept.
5. **`nle_verify`** (§9.3) and the R < 4.5 fix.

6. The scaled patch pushed as an EMC2 branch (`dev-nle-scaled`) rather than a file in this repository.

11.2 Then, if the package is wanted

M1 Python package skeleton with the RDM example (cheap to train, exact control); M2 the DDMnle example and the ONNX graph; M3 converter move and presets; M4 docs, container, CI. Each milestone ends with `nle_verify` passing on a fresh machine.

11.3 Decisions AH needs to make next

1. **Build the Python package, or freeze the scripts** (option B vs C). My recommendation is B; C is defensible if nobody will own the Python side.
2. **No separate R package**: agree that the R side lives in EMC2 `dev-nle`. If AH wants the NN layer kept out of EMC2 proper for the release, the alternative is a small `EMC2nle` R package that *depends* on EMC2 and holds `register_nn_model`, the kit and the loaders; that is a packaging choice, not new code, and it would keep CRAN EMC2 free of the C++ evaluators. I would still not call it nleFlow or put training in it.
3. **Which RDM is the worked example** (§3.1), and whether to commit the A = 0 flow's card.
4. **Approve one tux26 export run** for the DDMnle golden vectors (§2.4.2), and say where the checkpoints are.
5. **The rebase of `dev-nle` onto `dev`**: before or after M0.
6. The open conventions in §4.4 (SZ, s = 1, ±4 sd, tail mass, affine/decision-time status).

11.4 Deviation from the prompt

The prompt asks for the trailer `Co-Authored-By: Claude Sonnet 5.5`. This session ran on Fable 5.1, so the commit carries the trailer for the model that wrote it, plus the session link, rather than a false one.

---

12. Next stage (one model, one prompt)

Blocked on decisions 1–4 of §11.3 only for its last step; everything else can start. Model: **Opus 5.5** (or Fable), auto mode; this is EMC2-side, trap-dense work with exact acceptance tests, the kind this project assigns to Opus/Fable (`handover/ZERO_BOX_RETRAIN_PLAN.md`, "which model for which stage").

```
AUTO MODE REQUIRED: refuse to start unless this session is in auto mode.
Read handover/NLEFLOW_PLAN.md sections 1.1, 2.4, 7.1, 9.3 and 11.1 (nle repo, branch
nleflow-plan), then
handover/nle-methods/port/R/ddm_scaled.R, handover/emc2/emc2_dev-nle-scaled.patch, and in
the EMC2 dev-nle tree
R/nn_register.R (register_nn_model, nn_normalise_card, nn_native_spec), src/model_NN.h,
tests/testthat/test-nn-port.R.
Do milestone M0 of the plan, items 1, 2, 3 and 5, on a local EMC2 branch dev-nle-m0 off
142bd497 (the tarball in
handover/emc2, or the local dev-nle if present), installed into a NEW dedicated library;
never into ~/R/nle-lib.
Item 4 (DDMnle golden vectors) needs one export on tux26: do not run it; write the exact
command and stop at that
point if AH has not approved it. Do not rebase onto dev in this stage.
```

```
Acceptance, one line each: (a)
register_nn_model("port/export/ddm_sc_s1106_e245_weights.json", twin = DDM) returns
a flow_scaled registration with no scaffold; (b) its compiled likelihood equals
ddm_sc_density to <= 1e-11 over 200
prior draws in each of the six regimes of sbc/scaled_cell.R; (c) nn_golden_check() passes on
rdm_small and
cap256w_c4 against tests/testthat/golden and fails loudly on a perturbed golden row; (d) a
card without
card_version loads as version 0 and a card with card_version = 2 is refused by name; (e) on
R < 4.5 the shipped
artefacts either load (MANIFEST carries md5) or fail with a message that names the R
version; (f) Rscript
inst/scripts/nle_verify.R prints one PASS line per card and exits 0; (g) the full test-nn-*
suite passes.
Work with absolute paths, at most 8 cores, nothing on tux26, Gadi or AWS. Commit on the EMC2
branch and on
nleflow-plan (the plan's progress log, one line), push only those two branches, and end with
ONE next stage: M1 or
the golden-vector export, depending on AH's answers to section 11.3.
```

Progress log

- 2026-10-16 plan written (this file); probes run on the cloud instance as in §1.3; nothing on tux26, Gadi or AWS; no push to ampl-psych/EMC2.